Практическая работа №9. Конфигурирование сервера PostgreSQL.

Конфигурирование сервера позволит администратору баз данных определить настройки, подходящие для эксплуатации в конкретной организации для задач предметной области. Для того, чтобы осуществить конфигурации сервера на PostgreSQL достаточно выполнять команды в SQL Shell.

Для того, что вносить изменения в конфигурационный файл сервера необходимо знать к нему путь. Для этого достаточно выполнить команду show config_file; В данном случае система выдает полный путь доступа к конфигурационному файлу.

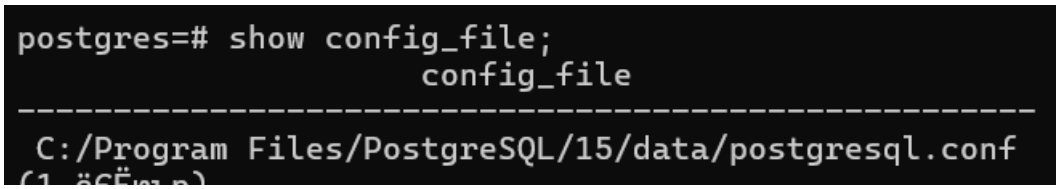


Рисунок 1 – Получение пути к файлу конфигурации сервера.

Самый простой способ просмотра и редактирования данного файла – с помощью блокнота (notepad ++).

Стоит заметить, что система предлагает перечитать параметры файла командой /q, либо подтвердить изменения можно командой SELECT pg_reload_conf(); в текущей версии легче согласиться с командной строкой:



Рисунок 2 – Завершение чтения файла

В текущей версии PostgreSQL по умолчанию установлена кодировка Russian_Russia.1251. Не трудно догадаться, что это windows 1251. При этом выполнение запросов в SQL Shell выглядит очень неудобно. Вместо информации появляется конфликт кодировки.

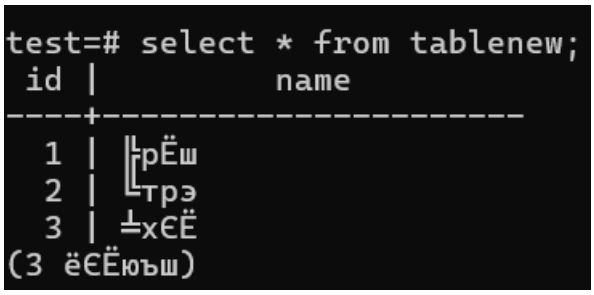


Рисунок 3 – Конфликт кодировки.

Для того, чтобы удостовериться в том, какая кодировка установлена на данный момент в конфигурационном файле достаточно выполнить следующий запрос:

```
test=# select sourceline, name, setting, applied from pg_file_settings;
 sourceline |            name             |       setting       | applied
------------+-----------------------------+---------------------+---------
         60 | listen_addresses            | *                   | t
         64 | port                        | 5432                | t
         65 | max_connections             | 100                 | t
        127 | shared_buffers              | 128MB               | t
        150 | dynamic_shared_memory_type  | windows             | t
        241 | max_wal_size                | 1GB                 | t
        242 | min_wal_size                | 80MB                | t
        444 | log_destination             | stderr              | t
        451 | logging_collector           | on                  | t
        461 | log_file_mode               | 0640                | t
        597 | log_timezone                | Europe/Moscow       | t
        711 | datestyle                   | iso, dmy            | t
        713 | timezone                    | Europe/Moscow       | t
        727 | lc_messages                 | Russian_Russia.1251 | t
        729 | lc_monetary                 | Russian_Russia.1251 | t
        730 | lc_numeric                  | Russian_Russia.1251 | t
        731 | lc_time                     | Russian_Russia.1251 | t
        734 | default_text_search_config  | pg_catalog.russian  | t
          3 | work_mem                    | 10MB                | t
```

Рисунок 4 – Вывод основных параметров сервера.

В данном запросе были выведены строки в конфигурационном файле, название параметров, их значение и возможность применимости параметра. Изменить настройки кодировки можно с помощью notepad++.

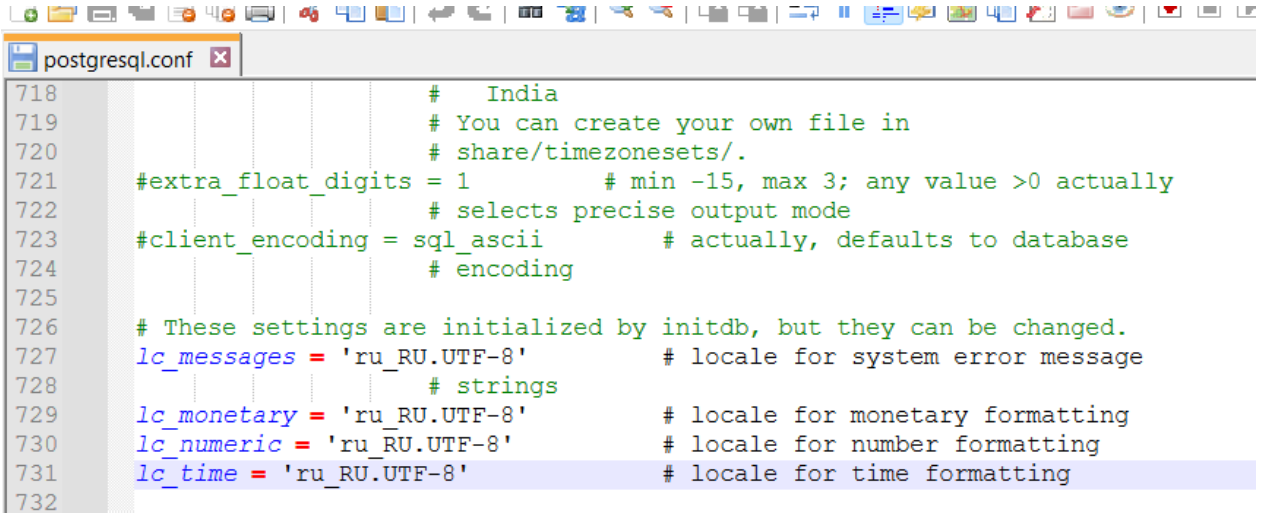


Рисунок 5 – Изменение конфигурации сервера в notepad++

Для проверки в SQL Shell были добавлены новые строки в базу после изменения кодировки и перезагрузки. В соответствии с новой кодировкой были выведены корректные значения:

```
test=# insert into tablenew (id, name) values (5, 'Марина');
INSERT 0 1
test=# select * from tablenew;
 id |         name
----+----------------------
  1 | ╠рЁш
  2 | ╚трэ
  3 | ╧хЄЁ
  4 | Наталья
  5 | Марина
```

Рисунок 6 – Вывод значений, добавленных с новой кодировкой.

Пример изменения параметра конфигурации в рамках транзакции.

В качестве примера рассматривается параметр work_mem. Данный параметр задает в PostgreSQL объем памяти, который будет необходим для внутренних операций сортировки и хеш-таблиц, прежде, чем будут задействованы временные файлы на диске. В частности назначить параметру значение можно с помощью конструкции ALTER SYSTEM… В данном случае изменения будут вноситься в первую очередь в файл postgresql.auto.conf, данный файл имеет больший приоритет, чем postgresql.conf.

```
postgres=# alter system set work_mem to '10MB';
```

Рисунок 7 – Изменение параметра с помощью запроса.

Теперь можно посмотреть информация об этом параметре с помощью следующего запроса:

```
postgres=# select name, setting, unit, boot_val, reset_val, source, sourcefile, sourceline,
 pending_restart, context from pg_settings where name='work_mem'\gx
-[ RECORD 1 ]---+---------
name            | work_mem
setting         | 4096
unit            | kB
boot_val        | 4096
reset_val       | 4096
source          | default
sourcefile      |
sourceline      |
pending_restart | f
context         | user
```

Рисунок 8 – Отображение параметров для work_mem.

В примере выше мы использовали расширенный режим (в конце запроса \gx), поэтому табличка перевёрнута. Разберём колонки:

- name — имя параметра;
- setting — текущее значение;
- unit — единица измерения;
- boot_val — значение по умолчанию (жёстко задано в коде postgresql);
- reset_val — если перечитаем конфигурацию, то применится это значение;

- source — источник, это значение по умолчанию;
- sourcefile — если бы источником был конфигурационный файл, то тут был бы указан этот файл;
- sourceline — номер строки в этом файле;
- pending_restart — параметр изменили в конфигурационном файле и требуется перезапуск сервера. У нас требуется всего лишь перечитать конфигурацию;
- context — действия, необходимые для применения параметра, может быть таким:
- internal — изменить нельзя, задано при установке;
- postmaster — требуется перезапуск сервера;
- sighup — требуется перечитать файлы конфигурации;
- superuser — суперпользователь может изменить для своего сеанса;
- user — любой пользователь может изменить для своего сеанса на лету.

Далее в качестве примера предложены две транзакции: в рамках первой транзакции производится откат после назначения параметру большей памяти. Во второй транзакции происходит успешное завершение, но с меньшими изменениями памяти в большую сторону. Таким образом, можно изменять параметры конфигураций не только с помощью ALTER SYSTEM, но и с помощью механизма транзакций.

Благодаря увеличению веса параметра work_mem можно следить за оптимизацией времени обработки запросов.

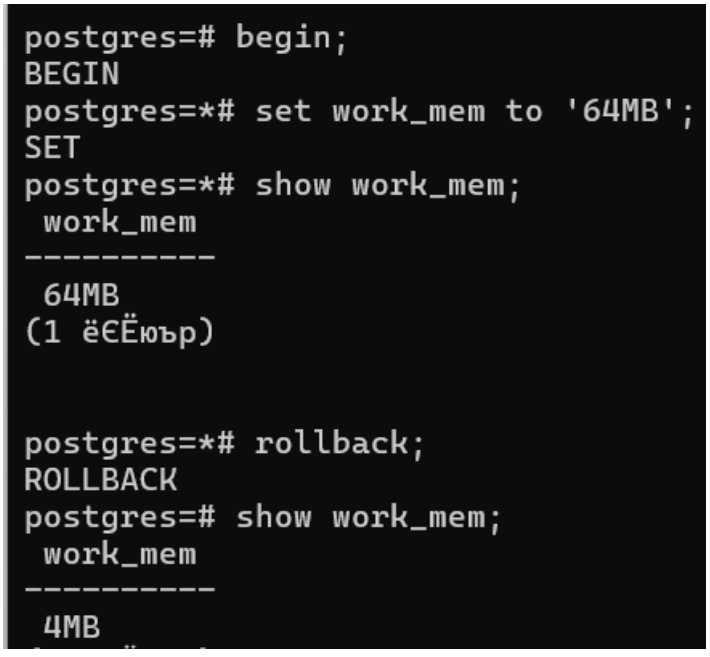


Рисунок 9 – Создание транзакции на изменение памяти для параметра с откатом.

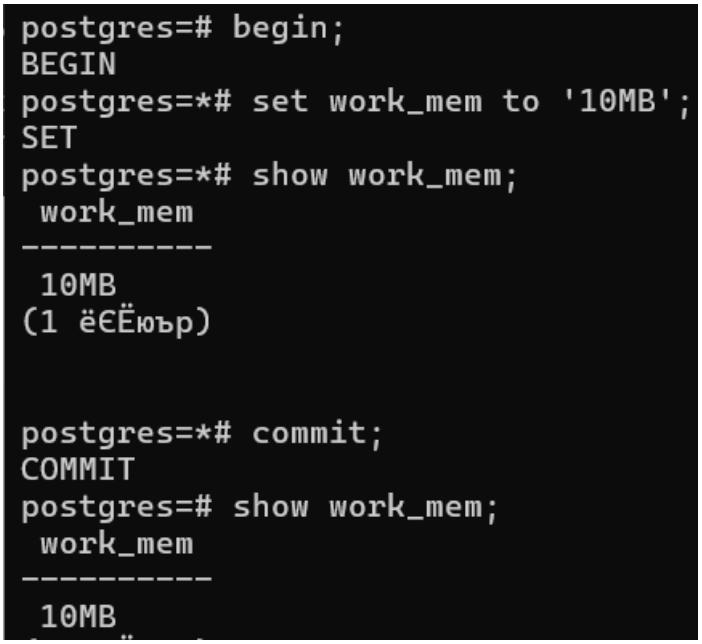


Рисунок 10 – Создание транзакции на изменение параметра с успешным завершением.

Задание.

Проверьте кодировку своей версии PostgreSQL, если она выводит значения некорректно, поменяйте ее. Выведите все атрибуты из pg_file_settings.

Измените параметр work_mem и выведите о нем информацию из pg_settings.

Создайте индекс в одной из таблиц с помощью запроса CREATE INDEX. После этого создайте запрос на выборку с учетом созданного индекса. Зафиксируйте время прохождения запроса.

Проведите 2 транзакции на изменение параметра work_mem. Увеличьте для успешной транзакции объем памяти для work_mem.

Проверите снова запрос для создания индекса с помощью CREATE INDEX и запроса на выборку. Сравните, изменилось ли время прохождения запроса.